Baseline reproducibility investigation

Brief scientific report

Prepared 5 June 2026

Context and initial concern

This investigation began with a reproducibility concern in the baseline semantic analysis pipeline. A colleague's local workflow pinned `numpy==1.26.4`, whereas the main `semantic-analysis` workflow used a modern locked stack with NumPy 2.x. The most visible discrepancy was in the baseline TriMAP result: the colleague's archived runs reported trustworthiness@10 values of 0.6007, 0.6012, and 0.5801, whereas the current `semantic-analysis` baseline reported 0.8771. The colleague's TriMAP plots also appeared visibly degraded. This raised two linked concerns: first, whether the NumPy version difference was causing the poor TriMAP outcome; and second, whether clustering via HDBSCAN was also being distorted.

A key structural finding from code inspection is that HDBSCAN is run on the embedding/PCA representation rather than on the 2D TriMAP coordinates. Poor TriMAP visualisation therefore does not automatically imply poor clustering assignments.

Experiment 1: locked modern environment with fixed embeddings

This experiment used the checked-in `semantic-analysis` code, the pinned `requirements-lock.txt`, and the cached `embeddings_large.parquet`, with three independent runs under seeds 42, 43, and 44.

Main findings:

- Clustering was perfectly stable across all three runs.
- Pairwise ARI, AMI, NMI, noise-status agreement, and co-cluster Jaccard were all equal to 1.0.
- Every run produced 3 clusters with the same noise fraction, 0.0698.
- Silhouette and evaluation silhouette were effectively unchanged across runs, at about 0.6533 and 0.1245 respectively.
- TriMAP trustworthiness varied only modestly, from 0.8677 to 0.8763.

Interpretation: with embeddings held fixed and the environment locked, the baseline clustering pipeline was highly reliable.

Experiment 2: NumPy-1.x-style approximation on the same code and embeddings

This experiment reused the same `semantic-analysis` code and cached embeddings, but approximated the colleague's environment by reinstalling `numpy==1.26.4`, rebuilding `annoy==1.17.3` from source, and reinstalling `trimap==1.1.5`.

Main findings:

- Clustering again remained perfectly stable across seeds 42, 43, and 44.
- Pairwise ARI, AMI, NMI, noise-status agreement, and co-cluster Jaccard were again all 1.0.
- The clustering summary was essentially unchanged from Experiment 1: 3 clusters, noise fraction 0.0698, silhouette about 0.6533, and evaluation silhouette about 0.1245.
- TriMAP trustworthiness was only slightly lower than in the modern locked environment, ranging from 0.8622 to 0.8661.

Interpretation: NumPy 1.x, on its own, did not recreate the colleague's severe TriMAP degradation and did not introduce clustering instability.

Experiment 3: colleague-style reproduction using colleague code and cached embeddings

This experiment moved closer to the colleague's workflow by using the OneDrive codebase, the `run_all_preserve_env.sh` package recipe, and the colleague's own cached embeddings from archived runs A, B, and C.

Main findings:

- For each of the archived runs A, B, and C, the reproduced HDBSCAN labels matched the archived labels exactly.
- Relative to the archived labels, ARI, AMI, NMI, noise-status agreement, and co-cluster Jaccard were all 1.0 for each reproduced run.
- The reproduced pairwise instability across embeddings matched the archived pattern: A vs B ARI = 0.578, A vs C ARI = 0.957, and B vs C ARI = 0.572.
- This shows that the clustering instability was already encoded in the colleague's cached embeddings.
- However, the poor archived TriMAP trustworthiness was not reproduced. Reproduced trustworthiness was about 0.8661 for A, 0.8662 for B, and 0.8595 for C, compared with archived values of 0.6007, 0.6012, and 0.5801.
- The reproduced clustering summaries matched the archived results closely, including cluster counts, noise fractions, and silhouette diagnostics.

Interpretation: the clustering instability in the colleague's outputs is upstream of HDBSCAN and is consistent with differences already present in the cached embeddings. By contrast, the very poor archived TriMAP trustworthiness appears to reflect a separate environment- or runtime-specific issue that was not recreated from the archived code, package recipe, and embeddings alone.

Summary of likely causal channels

Taken together, the three experiments point to two distinct causal channels.

First, the cluster instability is most plausibly driven by upstream embedding variation. Once the colleague's archived embeddings were supplied, the archived HDBSCAN outputs were recovered exactly, including the same cross-run disagreement structure.

Second, the archived TriMAP failure appears to be a separate reproducibility problem. It was present in the colleague's stored JSON outputs, but it was not reproduced either in the NumPy-1.x approximation on the main codebase or in the closer colleague-style reproduction using the archived embeddings. This suggests some additional machine-specific or runtime-specific factor in the TriMAP stack, plausibly involving native-package build conditions or other untracked environmental details.

Importantly, the evidence does not support the hypothesis that the NumPy 2.x versus 1.x difference is the main causal explanation. Moving from the modern locked environment to a NumPy-1.x-style environment left the clustering outcome unchanged and only slightly reduced TriMAP trustworthiness.

Conclusion

Across three progressively closer experiments, the baseline clustering pipeline was stable whenever embeddings were held fixed. The main reproducibility problem for clustering therefore appears to lie upstream, in the generation of embeddings rather than in HDBSCAN itself. Separately, the colleague's extremely poor TriMAP trustworthiness appears to reflect an additional environment-specific failure mode that remains unreproduced. The most plausible overall interpretation is therefore not a single NumPy-version effect, but a combination of embedding-generation instability and a distinct TriMAP/runtime issue.